ARGUS Simulation Results

Federated learning, frame rate under training, and poisoning resistance

Prepared for the sponsor and team. Code cells are collapsed. Every number below is produced by the simulator in `sim/` on the `feat/fine-taxonomy` branch, and the notebook re-runs end to end.

```
Run 2026-10-04  |  branch feat/fine-taxonomy @ f6d0b73
```

Summary

Question	Short answer
Does FL beat a node training alone?	**Yes for small data and new cameras. No for a mature site using the shared model as-is.** Federated + a short local fine-tune has the best balanced accuracy at every data size (0.771 at 50 events per site, 0.879 at 5,000). Local-only catches up by 1,500 events.
What does a brand-new camera get?	The fleet model works on day one: **0.75 balanced accuracy and 0.72 anomaly recall with zero local events**, against 0.10 untrained. Local-only needs about 150 events to match the accuracy and about 500 to match the anomaly recall.
Does detection keep running during training?	**Yes on every Hailo build tested:** pi5-hailo, the pipelined build and pi5-hailo-fine all stayed above 12 FPS during a real Flower round. It **does not hold 12 FPS** if the threat model runs on the CPU (10.3 FPS). Single-run PC measurements, not Pi measurements.
Can one bad camera poison the fleet?	**Yes with today's FedAvg server.** One crashed node destroys the model (0.00), and a boosted hostile node cuts anomaly recall from 0.64 to 0.08. robust.py blocks the crash, noise, sign-flip and heavily boosted attacks with no false rejections, but **a moderately boosted (x2.5) or quiet label flip gets through it.** Plain trimmed mean handled both boosted label flips better.

What is being simulated

The federated model is not the detector. YOLO stays frozen and finds person / vehicle / animal / package. The federated part is a small head (205,445 parameters, 19 inputs) that decides whether a detection is **routine for this site** or an **anomaly**. A delivery van at 14:00 is routine, and the same van at 03:00 is not.

Three simulated camera sites have different traffic and different rules for what counts as abnormal:

Site	Mostly sees	What is abnormal there
driveway	vehicles, residents	anything at night, loitering
backdoor	packages, staff	any vehicle, at any hour
yard	animals	people at night

The two headline metrics:

- **Balanced accuracy:** average accuracy across the 5 labels, so rare labels count as much as common ones.
- **Anomaly recall:** of the events that really were abnormal, the share the head flagged. This is the security number.

All accuracy results are averaged over 3 random seeds. Error bars show the spread between seeds.

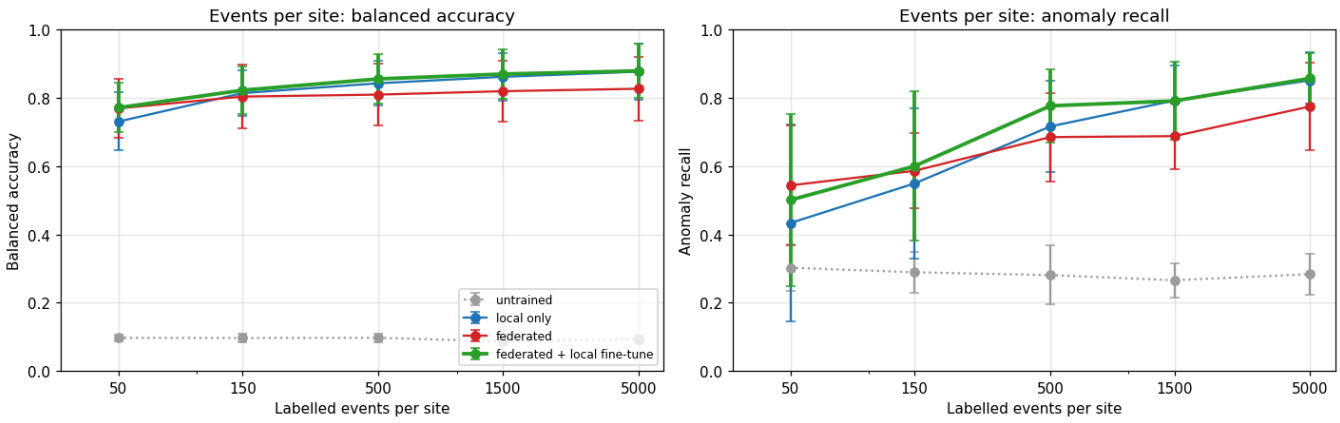
1. Does federated learning beat a node training alone?

This is the sponsor's question. Each site's head is obtained four ways:

Arm	How the head is trained
untrained	random weights (the floor)
local only	trains on its own events only, and never shares
federated	FedAvg across the 3 sites, 6 rounds; every site runs the same global model
federated + local fine-tune	the federated model, then a short training pass on the site's own events

Local-only gets the same number of local training epochs as federated + fine-tune ends up with, so it does not lose by training less. The federated arms still learn from 3 sites' data instead of 1; that is the point of federating.

1a. Same amount of data at every site



`Balanced accuracy (mean of sites and seeds)`

arm n_per_site	untrained	local only	federated	federated + local fine-tune
50	0.097	0.731	0.769	0.771
150	0.097	0.814	0.804	0.822
500	0.097	0.843	0.810	0.855
1500	0.087	0.862	0.819	0.870
5000	0.093	0.877	0.827	0.879

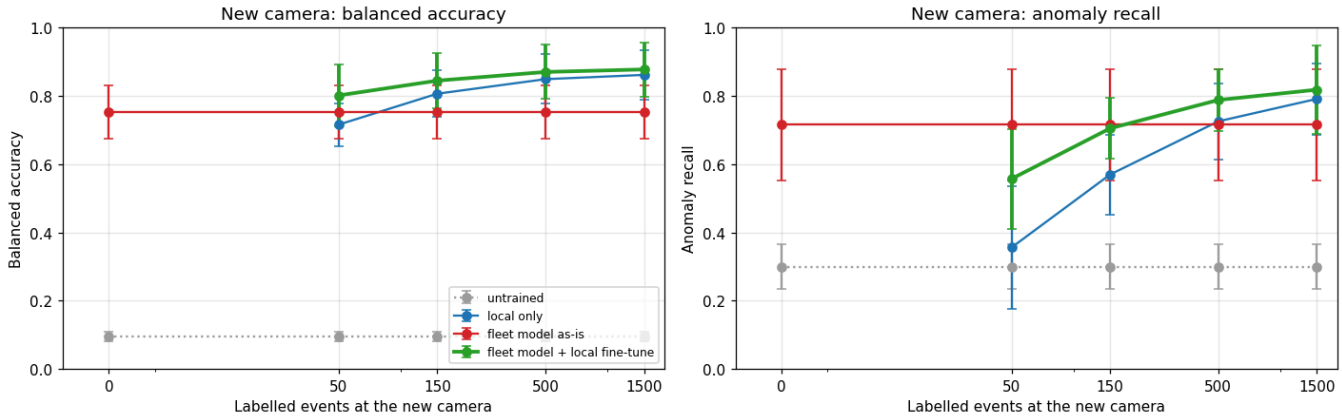
`Anomaly recall`

arm n_per_site	untrained	local only	federated	federated + local fine-tune
50	0.303	0.433	0.544	0.501
150	0.289	0.549	0.586	0.600
500	0.281	0.716	0.685	0.777
1500	0.266	0.792	0.688	0.791
5000	0.284	0.851	0.775	0.857

arm site	untrained	local only	federated	federated + local fine-tune
backdoor	0.101	0.757	0.698	0.759
driveway	0.084	0.874	0.832	0.895
yard	0.106	0.897	0.898	0.912

1b. A new camera joins the fleet

Leave-one-site-out: the fleet trains on two sites (3,000 events each). The third site gets a camera that the fleet model has **never seen**, with only `local n` events of its own (0 means it was installed today). This is the day-one experience of a new customer.



arm	untrained	local only	fleet model as-is	fleet model + local fine-tune
local_n				
0	0.096	NaN	0.752	NaN
50	0.096	0.716	0.752	0.801
150	0.096	0.806	0.752	0.845
500	0.096	0.849	0.752	0.870
1500	0.096	0.861	0.752	0.877

arm	untrained	local only	fleet model as-is	fleet model + local fine-tune
local_n				
0	0.299	NaN	0.715	NaN
50	0.299	0.356	0.715	0.557
150	0.299	0.570	0.715	0.705
500	0.299	0.725	0.715	0.788
1500	0.299	0.791	0.715	0.818

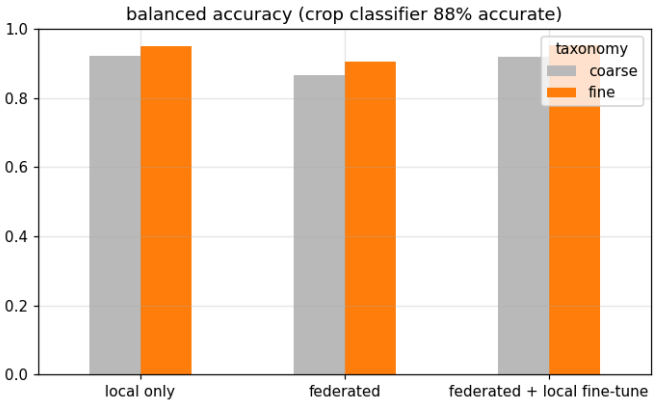
1c. Fine taxonomy (carrier, make, package size) and the household-car problem

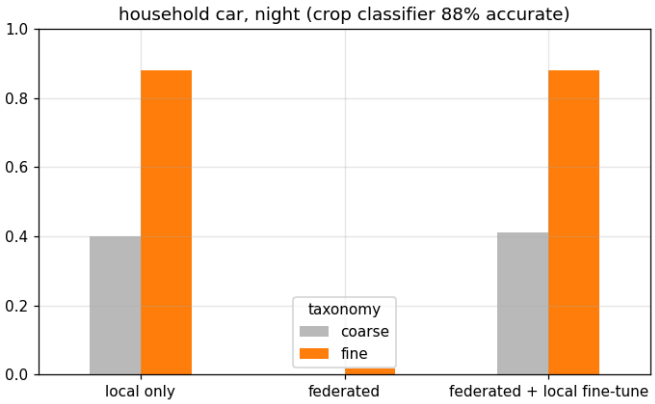
This re-runs `sim/compare_taxonomy.py` across 4 sites (adding `curbside`) and adds the fine-tune arm. "Household car, night" is the resident's own car arriving late. A shared model cannot know whose car it is, so this slice tests whether site-specific knowledge survives federation.

classifier_acc	taxonomy	arm	balanced accuracy	household car, night	stranger car, night	carrier truck, out of window	usps regular mail
0.76	**coarse**	**federated**	0.868	0.000	0.963	0.817	0.753
		federated + local fine-tune	0.921	0.412	0.663	0.432	0.900
		local only	0.922	0.400	0.617	0.492	0.924
	fine	**federated**	0.897	0.035	0.918	0.928	0.913
		federated + local fine-tune	0.944	0.825	0.712	0.743	0.940
		local only	0.932	0.811	0.635	0.690	0.916
0.88	**coarse**	**federated**	0.868	0.000	0.963	0.817	0.753
		federated + local fine-tune	0.921	0.412	0.663	0.432	0.900
		local only	0.922	0.400	0.617	0.492	0.924
	fine	**federated**	0.906	0.018	0.890	0.961	0.924
		federated + local fine-tune	0.953	0.881	0.715	0.833	0.935
		local only	0.949	0.881	0.789	0.854	0.938
1.00	**coarse**	**federated**	0.868	0.000	0.963	0.817	0.753
		federated + local fine-tune	0.921	0.412	0.663	0.432	0.900
		local only	0.922	0.400	0.617	0.492	0.924
	fine	**federated**	0.913	0.014	0.944	0.985	0.986
		federated + local fine-tune	0.968	1.000	0.796	0.948	0.980
		local only	0.961	0.971	0.760	0.960	0.977

Validation events per slice, summed over 4 sites (mean per seed). Small slices swing a lot:

	events
n: household car, night	20
n: stranger car, night	32
n: carrier truck, out of window	111
n: usps regular mail	120





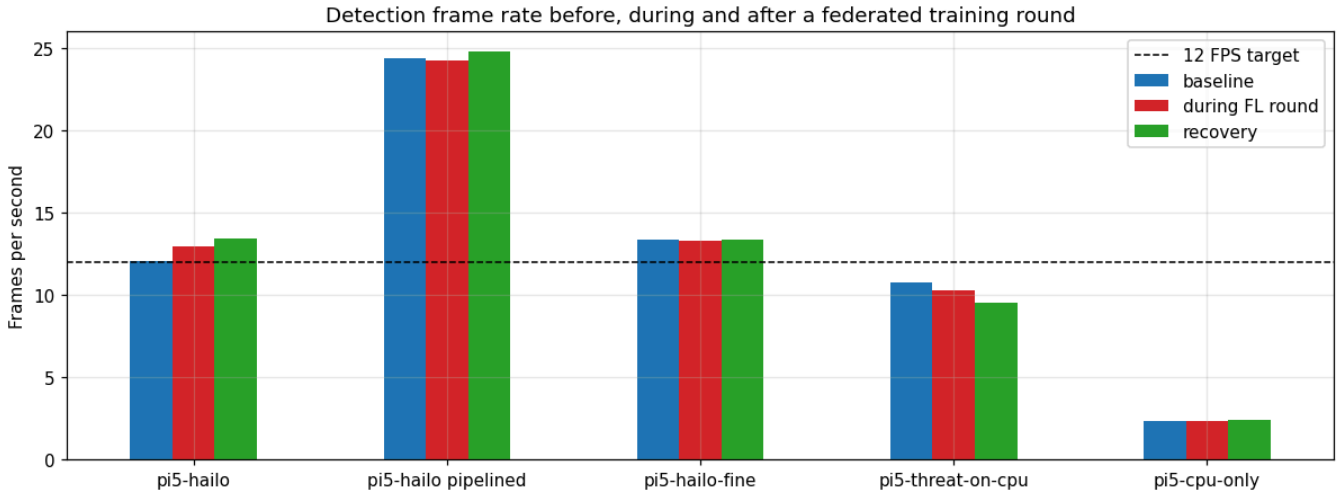
2. Does a node keep detecting while it trains?

`sim/run_node.py` runs the detection loop at 12 FPS. A **real Flower round** (gRPC, separate OS processes, 5 rounds x 12 epochs on 20,000 events) runs alongside it, pinned to 4 cores like a Pi 5. Frame rate is measured in three phases: before, during, and after training.

Configuration	Meaning
pi5-hailo	target build: YOLO + threat model on the Hailo, serial loop
pi5-hailo pipelined	same, with the Hailo overlapped with CPU work
pi5-hailo-fine	target build plus the carrier/make/size crop classifier
pi5-threat-on-cpu	risk case: the threat model never gets compiled for the Hailo
pi5-cpu-only	fallback: the Hailo is missing or fails its probe

Hailo and CPU stage times are budgets from Proposal Rev 2.2 Table 11. They are **not Pi measurements**.

configuration	baseline	fl	recovery	change during training	p95 ms during training	holds 12 FPS while training
pi5-hailo	12.08	12.98	13.48	7.5%	106.7	yes
pi5-hailo pipelined	24.42	24.27	24.81	-0.6%	45.8	yes
pi5-hailo-fine	13.37	13.34	13.39	-0.2%	75.6	yes
pi5-threat-on-cpu	10.81	10.29	9.57	-4.8%	125.1	NO
pi5-cpu-only	2.37	2.38	2.39	0.4%	456.8	NO



3. Can one bad camera poison the fleet?

A 6-camera fleet (2 per site) trains for 6 rounds. One driveway camera sends a bad update **every round**. The attacks:

Attack	What the bad camera sends
none	honest update (control)
NaN (diverged node)	a crashed or diverged trainer: all NaN
random noise	garbage weights
sign flip x4	its real update reversed and amplified, to drag the model backwards
label flip (hide anomalies)	trained honestly, but on data where every anomaly is labelled routine (a quiet, targeted attack)
label flip, boosted x2.5	the same, scaled just under robust.py's norm limit (a careful attacker)
label flip, boosted x6	the same, scaled up to take over the average (model replacement)

Server-side defences:

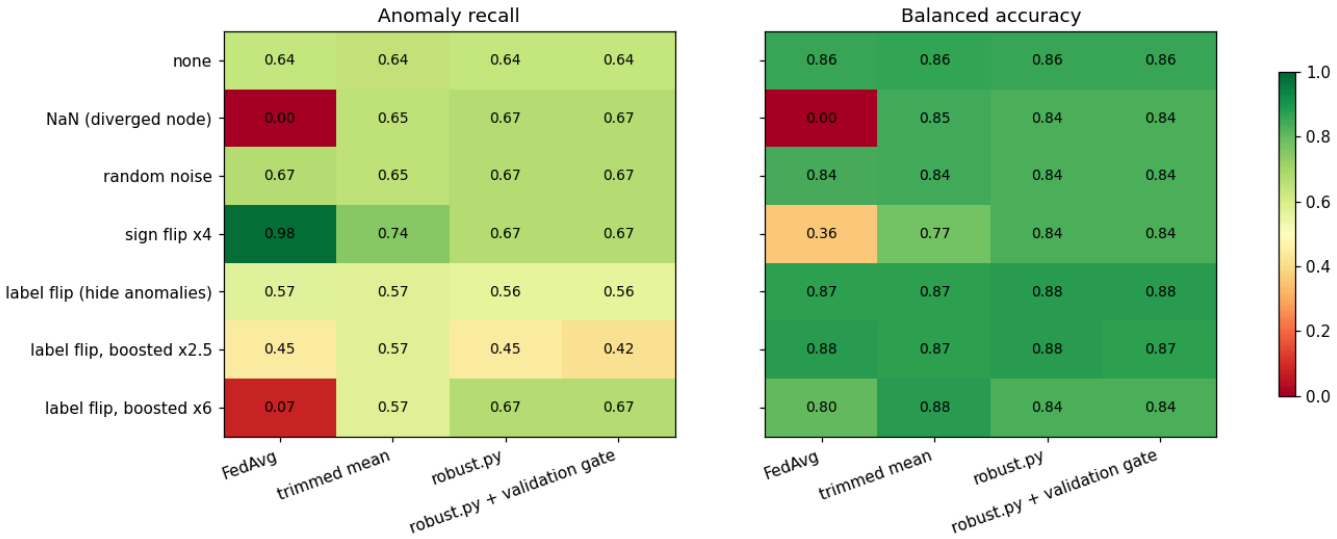
- **FedAvg:** what the Flower server runs today.
- **Trimmed mean:** drop the highest and lowest value of each weight, then average.
- **robust.py:** `src/federated/robust.py` with its shipped default settings. Written and tested, but **not yet wired into the server**.
- **robust.py + validation gate:** also refuses a new model whose balanced accuracy drops on a small server-side holdout (600 events the nodes never train on and the final score never uses).

`Anomaly recall of the fleet model after 6 rounds (0.000 = model destroyed)`

strategy attack	FedAvg	trimmed mean	robust.py	robust.py + validation gate
none	0.637	0.642	0.635	0.635
NaN (diverged node)	0.000	0.649	0.669	0.669
random noise	0.665	0.649	0.669	0.669
sign flip x4	0.978	0.743	0.669	0.669
label flip (hide anomalies)	0.568	0.573	0.560	0.560
label flip, boosted x2.5	0.448	0.575	0.449	0.416
label flip, boosted x6	0.075	0.573	0.669	0.669

`Balanced accuracy of the fleet model after 6 rounds (0.000 = model destroyed)`

strategy attack	FedAvg	trimmed mean	robust.py	robust.py + validation gate
none	0.859	0.862	0.859	0.859
NaN (diverged node)	0.000	0.847	0.836	0.836
random noise	0.839	0.844	0.836	0.836
sign flip x4	0.356	0.772	0.836	0.836
label flip (hide anomalies)	0.873	0.872	0.875	0.875
label flip, boosted x2.5	0.877	0.873	0.877	0.871
label flip, boosted x6	0.798	0.875	0.836	0.836



Rounds (of 6) in which robust.py rejected the bad camera, and honest updates it wrongly rejected (of 30):

	attacker_rejected_rounds		honest_rejections	
strategy	**robust.py**	**robust.py + validation gate**	**robust.py**	**robust.py + validation gate**
attack				
none	0.0	0.0	0.0	0.0
NaN (diverged node)	6.0	6.0	0.0	0.0
random noise	6.0	6.0	0.0	0.0
sign flip x4	6.0	6.0	0.0	0.0
label flip (hide anomalies)	0.0	0.0	0.0	0.0
label flip, boosted x2.5	0.0	0.0	0.0	0.0
label flip, boosted x6	6.0	6.0	0.0	0.0

Findings and recommendations

1. Federated learning vs local-only

- **The shared model alone trails a mature site.** Federated balanced accuracy rises slowly (0.769 at 50 events per site to 0.827 at 5,000), while local-only climbs to 0.877. The sites disagree about what is abnormal, and one averaged model cannot hold every site's rule. The cost falls mostly on the back door (0.698 federated vs 0.757 local).
- **Federation wins when data is scarce.** At 50 events per site the federated model beats local-only on balanced accuracy (0.769 vs 0.731) and on anomaly recall (0.544 vs 0.433).

- **Federated + local fine-tune has the best balanced accuracy at every size** (0.771 to 0.879). For anomaly recall it is best from 150 events upward, ties local-only at 1,500, and at 50 events the plain federated model is still better (0.544 vs 0.501). Fine-tuning should wait until a site has a few hundred labelled events.
- **Cold start is the strongest argument for FL.** A camera at a site the fleet has never seen gets 0.752 balanced accuracy and 0.715 anomaly recall on day one. Local-only needs about 150 events to match the accuracy and about 500 to match the recall.
- **Fine-tuning a new camera too early hurts recall.** With 50 or 150 local events, fine-tuning lifts balanced accuracy (0.801, 0.845) but drops anomaly recall below the untouched fleet model (0.557, 0.705 vs 0.715). It only wins on recall from 500 events (0.788).

1c. Fine taxonomy and site identity

Slices are small (household car at night: 20 validation events, stranger car at night: 32). Treat slice numbers as direction, not precision.

- The fine taxonomy raises balanced accuracy for every training method: federated +0.03 to +0.045, fine-tuned +0.023 to +0.047, depending on crop-classifier accuracy (76% to 100%).
- **A shared model cannot learn "this is the resident's car."** Federated scores 0.01 to 0.04 on household cars at night with the fine taxonomy. Local fine-tuning restores it to 0.83 to 1.00.
- **Trade-off:** fine-tuning makes a site forget fleet knowledge about rare events. With the 88% classifier, "stranger car at night" falls from 0.89 (federated) to 0.72 after fine-tuning, and "carrier truck out of its window" from 0.96 to 0.83. Fine-tuning needs a guard (fewer epochs, keep the fleet model's anomaly flag as a second opinion, or regularize toward fleet weights).

2. Frame rate while training

- **All three Hailo builds held 12 FPS during a real Flower round:** pi5-hailo 12.98, pipelined 24.27, pi5-hailo-fine 13.34. Training a 205K-parameter head on the CPU does not stop detection when the Hailo does the heavy work.
- **Below target:** pi5-threat-on-cpu (10.29 during training, 10.81 before) and the CPU-only fallback (about 2.4).
- **Read the percentages with care.** pi5-hailo went UP 7.5% during training and kept rising in recovery (13.48), which is background load on this PC, not a training effect. Earlier runs measured -8.1% and +0.1%. Which builds pass or fail 12 FPS was consistent across runs; the real numbers will come from a Pi 5.

3. Poisoning resistance

- **FedAvg (what the server runs today) is fragile.** One NaN node wipes the fleet model (0.000). A sign-flipped update drops balanced accuracy to 0.356 while recall jumps to 0.978 because the model flags nearly everything, which makes alerts useless. A x6 boosted "hide anomalies" update cuts recall from 0.637 to 0.075.

- **robust.py stops the loud attacks.** It rejected the bad camera in 6 of 6 rounds for NaN, noise, sign flip and the x6 boosted label flip, and rejected 0 of 30 honest updates in every scenario. With no attack it costs nothing (0.859 vs 0.859).
- **Moderate and quiet label flips get through robust.py.** The x2.5 boosted flip was never rejected and pulled recall to 0.449 (0.416 with the validation gate). The unboosted flip lowered recall from 0.64 to 0.56 under every defence.
- **Plain trimmed mean did better on label flips** (0.575 and 0.573 recall on x2.5 and x6) but lets a sign flip skew the model (balanced accuracy 0.772, recall 0.743).
- **The validation gate added nothing here.** It never rejected an update, because it checks balanced accuracy only, and balanced accuracy barely moves under a label flip (0.871 to 0.877).

Recommendations

1. **Ship federated + per-site fine-tune, start new cameras on the fleet model, and do not fine-tune before about 500 labelled events.** Guard against forgetting rare fleet-learned anomalies.
2. **Wire an aggregation defence into `central_server/fl_aggregator.py` before any multi-site trial.** Consider robust.py's rejection step followed by a trimmed mean instead of a plain mean, since each covers the other's gap in Section 3. This combination is untested.
3. **Make the validation gate check anomaly recall (plus the loss tie-break).** Untested hypothesis: it is the cheapest defence against label flips, since balanced accuracy alone did not move.
4. **Keep the threat model on the Hailo.** The CPU threat build is the only accelerated build under 12 FPS.
5. **Replace the Table 11 budgets with Pi 5 + Hailo measurements** (`scripts/benchmark_classes.py`, then `sim/profiles.py`) and re-run Section 2 on the Pi.

What this does not show

- **Synthetic events.** Site traffic, delivery windows, and anomaly rules are simulation assumptions. No real ARGUS footage is used. The 2026-09-20 hardware run (322 labelled events, balanced accuracy 0.75) is the only real-data point so far.
- **Frame-rate budgets, not Pi measurements.** The stage times come from Proposal Rev 2.2 Table 11. No Pi 5 + Hailo benchmark exists in the repo yet.
- **This machine's cores, not a Pi's.** Pinning to 4 cores matches the core count, not the per-core speed.
- **In-process federation for the accuracy studies.** It uses the same FedAvg maths as the Flower server, without the network. Section 2 uses the real Flower gRPC path.
- **Attacks are simple.** A patient attacker who stays just under the norm limit and agrees with the consensus direction is harder to stop. Section 3 shows the quiet label flip is already one of these.